

Lab 4 GPT-based Text-to-Speech System

Olena Azarova, Igor Ivanyshyn

1 Introduction

The goal of this lab was to complete and train a GPT-based Text-to-Speech (TTS) model using PyTorch. The homework involved implementing the training pipeline, training a GPT-2 style autoregressive model for audio token generation, and evaluating inference under different sampling strategies.

The model receives text tokens as input and generates discrete audio codec tokens, which are later decoded into waveform audio using a neural codec.

2 Model Architecture

The implemented system consists of the following components:

- Tokenizer for converting text into token IDs.
- GPT-2 style transformer decoder.
- Audio codec tokenizer for discrete speech tokens.
- Codec decoder for reconstructing waveform audio.

The model learns the mapping:

$$\text{Text Tokens} \rightarrow \text{Audio Tokens}$$

The generated audio tokens are then converted into waveform audio using the codec decoder.

3 Training Pipeline

The training pipeline was implemented using PyTorch.

The main training stages include:

1. Loading and preprocessing training data.
2. Tokenizing text inputs.
3. Encoding audio into codec tokens.
4. Forward pass through GPT-TTS model.
5. Computing autoregressive cross-entropy loss.
6. Backpropagation and optimizer update.

The optimizer used during training was AdamW. The model was trained for several epochs on GPU.

4 Training Procedure

The training loop performs the following operations for each batch:

- Move batch tensors to device.
- Compute model outputs.
- Calculate training loss.
- Perform backpropagation.
- Update model weights.

Average loss was tracked after each epoch.

5 Inference

Inference was performed by generating audio codec tokens autoregressively from text input.

The generation function supports configurable sampling parameters:

- Temperature
- Top-k sampling
- Maximum generated tokens

The following sampling strategies were tested:

Strategy	Temperature	Top-k
Greedy	0.3	20
Balanced	0.7	50
High-randomness	1.0	100

Table 1: Sampling strategies used during inference

5.1 Inference Examples

Several texts were synthesized during evaluation:

- “The quick brown fox jumps over the lazy dog.”
- “It’s simple, evocative, and arguably the most peaceful way to demonstrate that you don’t need many words to tell a full story.”
- “The cat slept.”

For each generated sample:

- Mel spectrograms were visualized.
- Audio duration was measured.
- Generated token count was recorded.
- Audio playback was inspected manually.

6 Qualitative Analysis

Different sampling strategies produced noticeably different outputs.

6.1 Low Temperature

Lower temperature values produced:

- More stable generations
- Less diversity
- More repetitive outputs

6.2 High Temperature

Higher temperature values produced:

- More diverse outputs
- Greater variability
- Higher risk of instability or noise

7 Spectrogram Analysis

Mel spectrograms were generated for different sampling strategies during inference.

Figure 1 shows the spectrogram produced using a greedy sampling strategy. The generated audio appears more stable and deterministic, with clearer harmonic patterns and less variability compared to higher-temperature sampling methods.

Figure 2 shows the spectrogram produced using a balanced sampling strategy. The generated audio demonstrates relatively stable harmonic structures and smoother frequency transitions.

Figure 3 shows the spectrogram generated with higher randomness. The output is more diverse but also contains noisier and less stable regions.

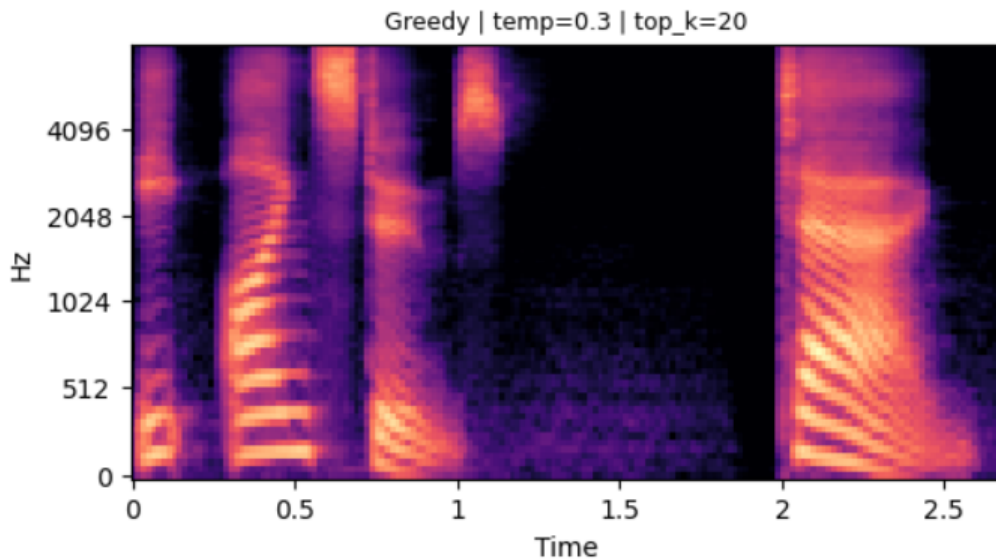


Figure 1: Greedy sampling strategy ($temperature = 0.7$, $top_k = 50$)

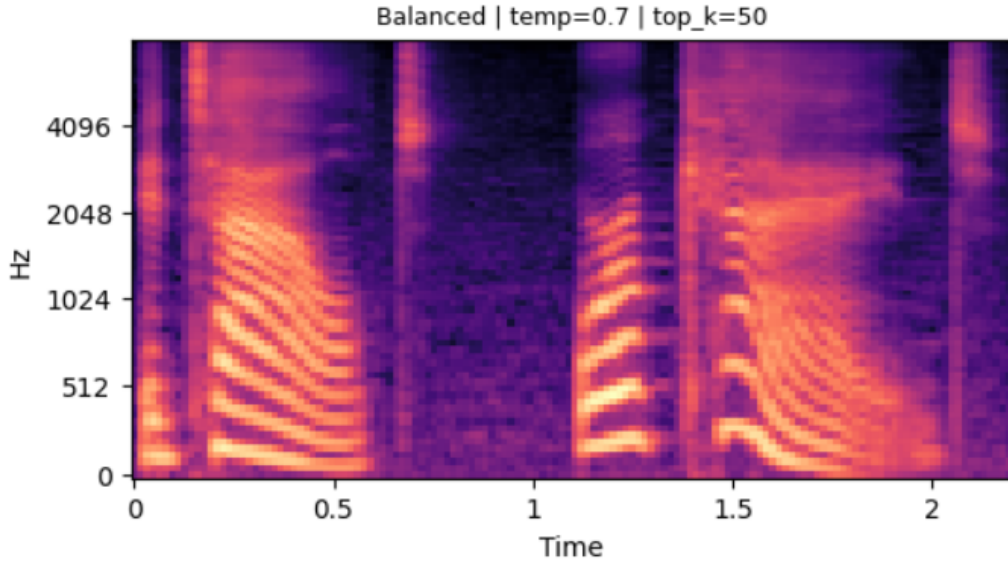


Figure 2: Balanced sampling strategy ($temperature = 0.7$, $top_k = 50$)

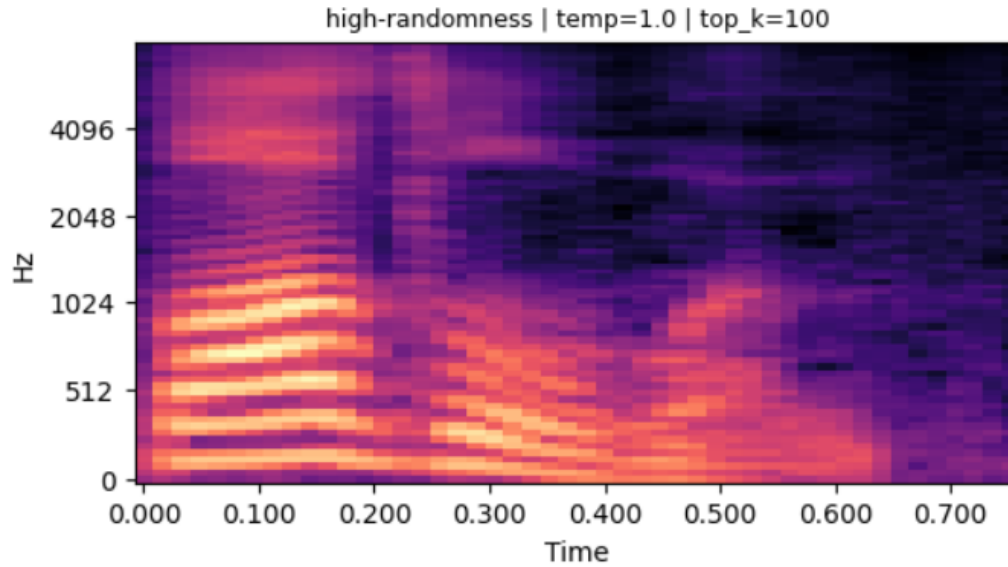


Figure 3: High-randomness sampling strategy ($temperature = 1.0$, $top_k = 100$)

8 Evaluation Metrics

The following objective metrics are planned for evaluation.

8.1 Character Error Rate (CER)

CER measures transcription accuracy between generated speech and reference text.

$$CER = \frac{S + D + I}{N}$$

where:

- S = substitutions
- D = deletions

- I = insertions
- N = number of characters

[Evaluation results will be added later.]

8.2 UTMOS

UTMOS estimates perceptual speech quality using a neural predictor model. Higher values indicate better predicted speech naturalness.

The average UTMOS score over the generated samples was:

$$\text{UTMOS} = 3.5215$$

8.3 SECS

SECS evaluates speaker similarity between generated and reference speech using cosine similarity between speaker embeddings. Higher values indicate stronger speaker similarity.

The average SECS score over the generated samples was:

$$\text{SECS} = 0.1921$$

8.4 CER

CER measures character-level transcription error between the target text and the ASR transcription of generated speech. Lower values indicate better intelligibility.

The average CER over the generated samples was:

$$\text{CER} = 0.8106$$

9 Future Improvements

Possible future improvements include:

- Larger training dataset
- Longer training schedule
- Better codec representation
- Improved sampling methods
- Beam search decoding
- Speaker conditioning
- Multi-speaker support

10 Conclusion

In this lab, we implemented and trained using PyTorch GPT-based Text-to-Speech system. The project demonstrated how autoregressive transformer models can generate audio codec tokens directly from text input.

Explored different sampling strategies during inference, showing the trade-off between generation stability and diversity.

Although objective evaluation metrics are still being computed, the system successfully generated intelligible speech samples and provided practical experience with modern TTS architectures.